

TP : TRAITEMENT D'IMAGES

Objectifs du TP

- Manipulation des images avec python.
- Stéganographie.
- Traitements de base.
- Détection de contour et formes.

La bibliothèque `matplotlib` offre la possibilité de manipuler des images.

On utilisera comme entête :

```
import numpy as np          # Calcul numérique
import matplotlib.pyplot as plt # Tracé
import PIL                  # Gestion des formats != png
```

On charge une image avec la fonction `plt.imread` :

```
mon_image = plt.imread("nom_image.png")
```

On obtient alors un tableau (numpy) à 3 dimensions :

- La première est la hauteur de l'image,
- la seconde est la largeur de l'image,
- la troisième est la profondeur correspondant au codage de l'image : pour une image au format `png` en couleur sans transparence¹, elle vaut 3, correspondant au codage Rouge Vert Bleu de l'image : des flottants entre 0 et 1 pour le format `png`, des d'entiers sur 1 octet (de 0 à 255) pour les images au format `jpg` ou `png`.

On récupère les dimension de l'image avec la méthode `shape` :

```
hauteur, largeur, _ = mon_image.shape
```

(la troisième coordonnée ne nous intéresse pas si on connaît à l'avance le type d'image).

On peut afficher l'image grâce à la fonction `imshow` de `matplotlib.pyplot` et l'enregistrer dans un fichier grâce à `plt.imsave` :

```
plt.imshow(mon_image)
plt.show()
plt.imsave('nom_fichier', mon_image)
```

Pour afficher/enregistrer une image en niveaux de gris, il faut ajouter un option `cmap`. Dans ce cas, un tableau à deux dimensions suffit (le niveau de gris pour chaque pixel).

```
plt.imshow(image, cmap='gray')
plt.imsave('nom_fichier', image, cmap='gray')
```

1. Pour une image ne niveau de gris, elle vaut 1 (seulement le niveau de gris) et pour une image avec transparence, elle vaut 4 car il faut rajouter un coefficient de transparence.

⚠ Le pixel à la i^{e} ligne (ordonnée) et la j^{e} colonne (abscisse) s'obtient avec `image[j, i]` (inversion).

```
>>> mon_image[2, 7]
array([ 0.76862746,  0.43921569,  0.29411766], dtype=float32)
```

Enfin, il est très facile de créer une « image vide » (qu'il faudra remplir avant de l'afficher!) :

```
nouvelle_image = np.empty((hauteur, largeur, 3)) # doubles parenthèses
nouvelle_image2 = np.empty(mon_image.shape)
```

Toutes les images obtenues seront enregistrées dans le répertoire de travail.

Stéganographie

Le principe est le suivant : pour une image dont les couleurs des pixels sont stockées sur un octet, on remplace les 4 bits de poids faibles (les 4 derniers) par les 4 bits de poids fort du pixel correspondant dans une autre image. On peut ainsi cacher une image à l'intérieur d'une autre (ou même n'importe quelle message à l'intérieur d'une image).

1. Avec quelle opération mathématique très simple peut-on transformer un entier entre 0 et 255 pour obtenir l'écriture décimale de ses 4 bits de poids faibles ? Quelle opération mathématique permet de transformer ce nombre en le nombre dont ce sont les 4 bits de poids forts (les autres valant 0) ?

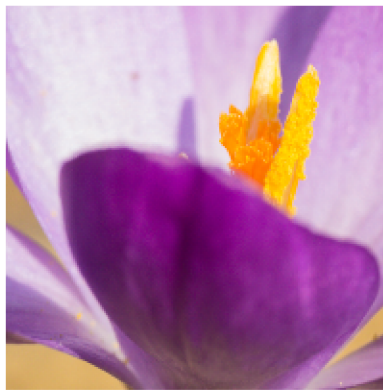
2. Trouver l'image cachée derrière l'image `paris.bmp`.



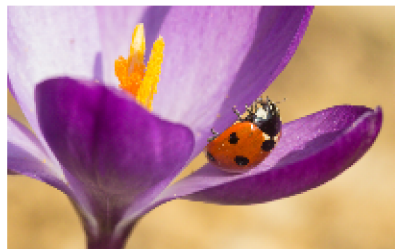
Transformations d'images



1. Réaliser un crop (découpage) de l'image en ne gardant que le carré de côté 200 pixel en haut à gauche. (Indication : utilisez le slicing!). Afficher et enregistrer le résultat.



2. Créer une image deux² fois plus petite obtenue en ne gardant qu'un pixel sur deux.



3. Convertir l'image en niveaux de gris en remplaçant les trois couleurs de chaque pixel par leur moyenne arithmétique (gris ssi $R=G=B$). Essayer aussi avec la luma³ donnée par la formule

$$Y = 0,299 \cdot R + 0,587 \cdot V + 0,114 \cdot B.$$



2. ou quatre ?

3. voir <https://fr.wikipedia.org/wiki/YUV>

4. Réaliser une symétrie d'axe vertical sur l'image. (Slicing autorisé!)



Filtres

Beaucoup de traitements d'images reposent sur de la convolution de matrices. Le principe est le suivant : on remplace chaque pixel de l'image par une combinaison linéaire de ce pixel et de ses voisins.

Pour cela, on se donne un **noyau** appelé masque de convolution, qui est une matrice contenant les coefficients correspondant au poids de chaque pixel voisin dans cette combinaison linéaire lorsque l'on centre la matrice sur le pixel à modifier.

Si, par exemple, K est une matrice 3×3 , et $I_{i,j}$ un pixel de notre image, alors on va le changer en

$$I'_{i,j} = \sum_{0 \leq k, \ell \leq 2} I_{i+k-1, j+\ell-1} K_{k,\ell}$$

Par exemple, avec le filtre $K = \begin{pmatrix} 0 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$, on obtient $I'_{i,j} = -I_{i,j-1} + I_{i,j}$.

1. Écrire une fonction `convol(I, K)` où K est un tableau à deux dimensions carré de taille $2p+1$ et I un tableau à au moins deux dimensions (par exemple une image) renvoyant un tableau de même format que I .

La convolution s'appliquera à tous les pixels sauf ceux étant à p pixels du bords pour être convenablement définie.

2. Un premier exemple simple : si on veut faire de la détection de contour simple, on essaye de savoir lorsqu'il y a un changement brusque d'intensité d'une couleur⁴ : d'où l'idée de considérer la variation d'intensité de couleur. Comme une image est discrète et non continue, la dérivée dans la direction horizontale ou verticale peut être approchée par

$$I_{i+1,j} - I_{i,j} \quad \text{ou} \quad I_{i,j+1} - I_{i,j} \quad \text{ou} \quad I_{i,j} - I_{i-1,j} \quad \text{ou} \quad I_{i,j} - I_{i,j-1}$$

Pour la quatrième, ça donne par exemple $K = \begin{pmatrix} 0 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$.

Les tester tous les quatre.

Pour parfaire notre détection de contour, tâchons de tenir compte des deux directions : si $I3$ est l'image obtenue par le 3^e filtre et $I4$ celle obtenue par le 4^e, créer l'image $I5$ telle que $I5_{i,j} = \sqrt{(I3_{i,j})^2 + (I4_{i,j})^2}$ (c'est très facile avec les tableaux numpy!!)

4. On pourra appliquer cela à une image (convertie) en noir et blanc, ou l'une des composantes RGB d'une image couleur, ou les trois composantes à la fois.



3. Ce genre de considérations peut amener à des filtres plus élaborés comme le filtre de Sobel (1970) qui se décompose en

$$\frac{dI}{dx} = \frac{1}{4} \begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{pmatrix} \quad \text{et} \quad \frac{dI}{dy} = \frac{1}{4} \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

L'implémenter sur le même principe que la question précédente.



On peut appliquer divers filtres pour obtenir d'autres effets du type :

- Lissage (flou par moyenne) :

$$\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}.$$

- Flou Gaussien (poids d'autant plus fort que le voisin est proche) :

$$\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} \quad \text{ou} \quad \begin{pmatrix} 1 & 1 & 2 & 1 & 1 \\ 1 & 2 & 4 & 2 & 1 \\ 2 & 4 & 8 & 4 & 2 \\ 1 & 2 & 4 & 2 & 1 \\ 1 & 1 & 2 & 1 & 1 \end{pmatrix}$$

(à normaliser en divisant par la somme des coefficients.)

- Filtres réhausseurs de contours (à l'inverse des précédents) :

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 10 & -1 \\ 0 & -1 & -1 \end{pmatrix} \quad \text{ou} \quad \begin{pmatrix} 1 & -2 & 1 \\ -2 & 5 & -2 \\ 1 & -2 & 1 \end{pmatrix} \quad \text{ou} \quad \begin{pmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

Détection de formes par la transformée de Hough ⁵

Le principe de la détection de droites par la transformée de Hough est le suivant : chaque droite du plan peut être décrite par deux paramètres (en coordonnées cartésiennes ou polaires). On va donc considérer un tableau à deux dimensions correspondant aux couples de paramètres de chaque droite et on va comptabiliser le nombre de pixel sur l'image se trouvant sur chacune des droites. On ne gardera au final que les droites contenant le plus grand nombre de pixels.

La méthode s'adapte sans problème à de la détection de cercles (3 paramètres pour le centre et le rayon) ou de courbes plus élaborées.

Pour parfaire la détection de droite, on commence par appliquer un filtre de détection de contour type Sobel sur une image convertie en niveaux de gris.

Le principe est donc le suivant :

- On choisit pour détecter des droites d'équation $y = ax + b$ sur l'image, de faire varier a entre a_{min} et a_{max} avec un pas Δa (dans un tableau A), et b entre b_{min} et b_{max} avec un pas Δb (dans un tableau B), ce qui conditionne la taille du tableau à deux dimensions appelé transformée de Hough de l'image que l'on notera `transformee`.
- Pour chaque point de coordonnées (x, y) sur l'image, que l'on peut par exemple supposées dans $[0, 1]^2$ (quitte à diviser par la largeur et la hauteur respectivement),
 - ★ Pour chaque valeur a ,
 - On calcule $b = y - ax$
 - On incrémente la case du tableau `transformee` correspondant à a et au b le plus proche dans B.

1. Écrire une fonction

```
transformee_Hough(image, A, B, distance, seuil)
```

prenant en argument `image` correspondant à une image en niveau de gris sur laquelle on aura déjà appliqué un filtre de détection de contour, et les paramètres correspondant à la description précédente (A et B sont des tableaux des valeurs possibles de a et b obtenus par exemple avec `linspace`) et renvoyant le tableau de la transformée de Hough de l'image comme décrit ci-dessus.

2. Pour éviter les détection multiples d'une même droite (points pas tout à fait alignés, droites épaisses...) et décide pour un maximum donné, d'éliminer ses voisins (par exemple sur une distance de 3 cases) ayant une valeur égale à au moins 90 % du maximum, en annulant leur valeur dans le tableau.

Écrire une fonction `nettoye(transformee)` qui renvoie une version nettoyée du tableau `transformee`.

3. Écrire une fonction `droites(image, A, B, distance, seuil)` qui renvoie la liste des paramètres a, b correspondant aux maxima de la transformée de Hough nettoyée.

Tester en superposant image et droites sur une même figure.

5. voir par exemple <http://www.i3s.unice.fr/~mh/RR/2004/RR-04.05-D.LINGRAND.pdf>
ou encore <http://documents.irevues.inist.fr/bitstream/handle/2042/2334/02.PDF%20TEXTE.pdf>